ASSIGNMENT-23.3

Roll no:2303A52074
Batch-37

Task 1 – Project Proposal

PROMPT

Generate a project proposal for an AI-based student attendance tracking system using facial recognition. The proposal should include objective, features, tools/technologies, and expected output. Keep the explanation simple, structured, and suitable for an academic assignment.

OUTPUT

Project Proposal: AI-Based Student Attendance Tracking System using Facial Recognition

1. Objective

The primary objective of this project is to develop and implement an automated student attendance tracking system using facial recognition technology. This system aims to replace traditional manual attendance methods, reducing errors, saving time, and providing an efficient, reliable, and secure way to record student presence in classrooms. The system will ensure accurate and real-time attendance data collection for academic purposes.

2. Features

The proposed system will include the following key features:

- **Automated Attendance Marking:** Students' attendance will be automatically marked upon entering the classroom by identifying their faces through a camera feed.
- **Student Enrollment Module:** A secure module for registering new students, capturing their facial data, and associating it with their academic records.
- **Real-time Monitoring:** Live camera feeds will be processed to identify students and update attendance records in real-time.
- **Attendance Reports:** Generation of detailed attendance reports (daily, weekly, monthly, by course) for faculty and administration.
- **User-Friendly Interface:** A simple and intuitive web-based or desktop application for managing student data, viewing attendance, and generating reports.
- **High Accuracy & Robustness:** The system will be designed to handle variations in lighting, student appearance (e.g., eyeglasses, hairstyles), and minor occlusions.
- **Security & Privacy:** Robust measures will be implemented to protect student facial data and personal information, complying with data privacy regulations.

3. Tools/Technologies

This project will utilize a combination of programming languages, libraries, and frameworks:

- **Programming Language:** Python
- **Facial Recognition Library:** OpenCV (for image processing and camera interaction) and dlib or FaceNet (for facial landmark detection and recognition).
- **Machine Learning Frameworks:** TensorFlow or PyTorch (for training and deploying facial recognition models).
- **Web Framework (for UI):** Flask or Django (for building the web application).
- **Database:** PostgreSQL or MySQL (for storing student profiles, facial embeddings, and attendance records).
- **Hardware:** High-resolution cameras (webcams or IP cameras).
- **Deployment:** Docker (for containerization) and potentially cloud platforms like AWS, Google Cloud, or Azure for scalability (optional).

4. Expected Output

Upon successful completion, the project is expected to deliver the following:

- **Fully Functional Attendance System:** A deployable software solution capable of accurately tracking student attendance using facial recognition.
- **Detailed Documentation:** Comprehensive project documentation, including system architecture, technical specifications, user manuals, and API documentation.
- **Trained Facial Recognition Model:** A robust and accurate pre-trained model for identifying registered students.
- **Attendance Database:** A well-structured database containing student information and attendance logs.
- **Reporting Interface:** An intuitive interface for generating and exporting various attendance reports.
- **Evaluation Report:** A report detailing the system's performance, accuracy, and limitations, along with potential future enhancements.

EXPLANATION

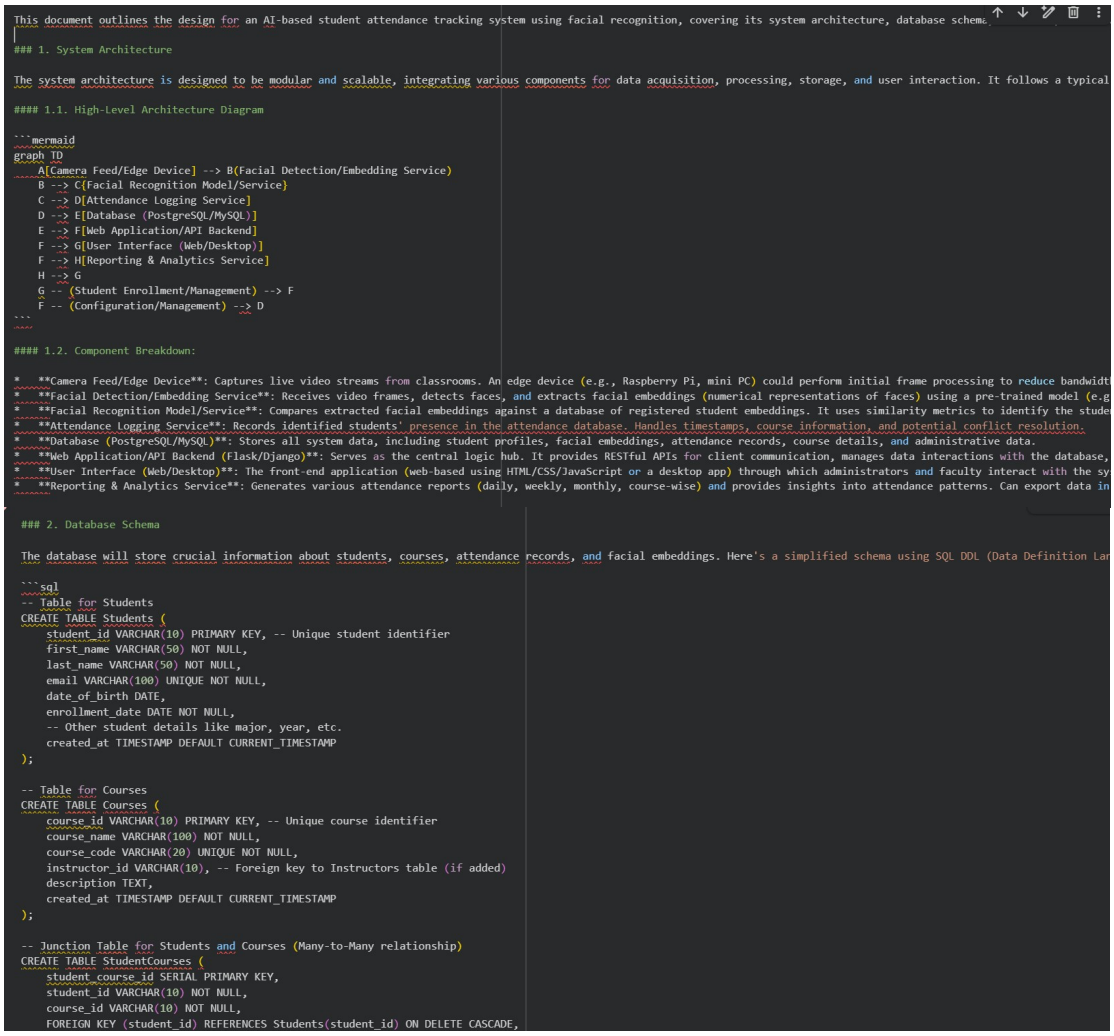
In this task, we designed a project proposal for an AI-based application by identifying a real-world problem. We defined the objective, features, tools, and expected output of the system. This task focuses on planning the project rather than implementation.

Task 2 – Application Design

PROMPT

Generate the application design output for an AI-based student attendance tracking system using facial recognition. Include system architecture, database schema, and UI wireframes. Provide structured output with clear explanations suitable for a student assignment..

OUTPUT



This document outlines the design for an AI-based student attendance tracking system using facial recognition, covering its system architecture, database schema

1. System Architecture

The system architecture is designed to be modular and scalable, integrating various components for data acquisition, processing, storage, and user interaction. It follows a typical

1.1. High-Level Architecture Diagram

```mermaid
graph TD
    A[Camera Feed/Edge Device] --> B(Facial Detection/Embedding Service)
    B --> C{Facial Recognition Model/Service}
    C --> D[Attendance Logging Service]
    D --> E[Database (PostgreSQL/MySQL)]
    E --> F[Web Application/API Backend]
    F --> G[User Interface (Web/Desktop)]
    F --> H[Reporting & Analytics Service]
    H --> G
    G -- (Student Enrollment/Management) --> F
    F -- (Configuration/Management) --> D
```

1.2. Component Breakdown:

*   **Camera Feed/Edge Device**: Captures live video streams from classrooms. An edge device (e.g., Raspberry Pi, mini PC) could perform initial frame processing to reduce bandwidt
*   **Facial Detection/Embedding Service**: Receives video frames, detects faces, and extracts facial embeddings (numerical representations of faces) using a pre-trained model (e.g
*   **Facial Recognition Model/Service**: Compares extracted facial embeddings against a database of registered student embeddings. It uses similarity metrics to identify the stude
*   **Attendance Logging Service**: Records identified students' presence in the attendance database. Handles timestamps, course information, and potential conflict resolution.
*   **Database (PostgreSQL/MySQL)**: Stores all system data, including student profiles, facial embeddings, attendance records, course details, and administrative data.
*   **Web Application/API Backend (Flask/Django)**: Serves as the central logic hub. It provides RESTful APIs for client communication, manages data interactions with the database,
*   **User Interface (Web/Desktop)**: The front-end application (web-based using HTML/CSS/JavaScript or a desktop app) through which administrators and faculty interact with the sy
*   **Reporting & Analytics Service**: Generates various attendance reports (daily, weekly, monthly, course-wise) and provides insights into attendance patterns. Can export data in

2. Database Schema

The database will store crucial information about students, courses, attendance records, and facial embeddings. Here's a simplified schema using SQL DDL (Data Definition Lar

```sql
-- Table for Students
CREATE TABLE Students (
    student_id VARCHAR(10) PRIMARY KEY, -- Unique student identifier
    first_name VARCHAR(50) NOT NULL,
    last_name VARCHAR(50) NOT NULL,
    email VARCHAR(100) UNIQUE NOT NULL,
    date_of_birth DATE,
    enrollment_date DATE NOT NULL,
    -- Other student details like major, year, etc.
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);

-- Table for Courses
CREATE TABLE Courses (
    course_id VARCHAR(10) PRIMARY KEY, -- Unique course identifier
    course_name VARCHAR(100) NOT NULL,
    course_code VARCHAR(20) UNIQUE NOT NULL,
    instructor_id VARCHAR(10), -- Foreign key to Instructors table (if added)
    description TEXT,
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);

-- Junction Table for Students and Courses (Many-to-Many relationship)
CREATE TABLE StudentCourses (
    student_course_id SERIAL PRIMARY KEY,
    student_id VARCHAR(10) NOT NULL,
    course_id VARCHAR(10) NOT NULL,
    FOREIGN KEY (student_id) REFERENCES Students(student_id) ON DELETE CASCADE,
```

```
-- Table for Facial Embeddings
CREATE TABLE FacialEmbeddings (
    embedding_id SERIAL PRIMARY KEY,
    student_id VARCHAR(10) UNIQUE NOT NULL, -- Each student has one primary embedding record
    embedding_vector BYTEA NOT NULL, -- Store facial embedding as a binary array
    image_path VARCHAR(255), -- Path to the reference image used for embedding
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    FOREIGN KEY (student_id) REFERENCES Students(student_id) ON DELETE CASCADE
);

-- Table for Attendance Records
CREATE TABLE Attendance (
    attendance_id SERIAL PRIMARY KEY,
    student_id VARCHAR(10) NOT NULL,
    course_id VARCHAR(10) NOT NULL,
    timestamp TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP,
    status VARCHAR(20) DEFAULT 'Present', -- 'Present', 'Absent', 'Late'
    -- Additional details like camera_id, entry_point, etc.
    FOREIGN KEY (student_id) REFERENCES Students(student_id) ON DELETE CASCADE,
    FOREIGN KEY (course_id) REFERENCES Courses(course_id) ON DELETE CASCADE
);

-- Table for Users (Admin/Faculty - for login and system management)
CREATE TABLE Users (
    user_id SERIAL PRIMARY KEY,
    username VARCHAR(50) UNIQUE NOT NULL,
    password_hash VARCHAR(255) NOT NULL,
    email VARCHAR(100) UNIQUE NOT NULL,
    role VARCHAR(20) NOT NULL DEFAULT 'Faculty', -- 'Admin', 'Faculty'
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);
```

3. UI Wireframes (Conceptual)

```
### 3. UI Wireframes (Conceptual)

Since direct image generation is not possible, I will describe the key screens and their functionalities as conceptual wireframes.

#### 3.1. Login Screen

*   **Elements**: University Logo, Username/Email Input Field, Password Input Field, "Remember Me" Checkbox, Login Button, "Forgot Password" Link.
*   **Functionality**: Allows administrators and faculty to securely log into the system.

#### 3.2. Dashboard (Faculty/Admin)

*   **Elements**: Navigation Bar (Home, Students, Courses, Attendance, Reports, Settings, Logout), Welcome Message, Quick Stats (e.g., "Today's Classes", "Students Present Now",
*   **Functionality**: Provides an overview of the system's status and quick access to main features.

#### 3.3. Student Enrollment/Management Screen

*   **Elements**: "Add New Student" Button, Search Bar (by ID, name), Student List/Table (with ID, Name, Email, Enrollment Date, Actions: Edit, Delete), Student Details Form (Fi
*   **Functionality**: Allows administrators to add new students, update their profiles, and upload images for facial embedding generation. Manage existing student records.

#### 3.4. Course Management Screen

*   **Elements**: "Add New Course" Button, Search Bar, Course List/Table (Course Code, Name, Instructor, Actions: Edit, Delete), Course Details Form (Course Code, Name, Instruct
*   **Functionality**: Enables faculty/admin to create, edit, and delete course records.

#### 3.5. Real-time Attendance Monitoring Screen

*   **Elements**: Live Video Feed from Camera, "Select Class/Course" Dropdown, List of Identified Students (Name, Student ID, Time Detected, Status: Present), List of Unidentifi
*   **Functionality**: Displays the live camera feed and shows students as they are recognized. Allows faculty to monitor attendance in real-time and manually mark absent studen

#### 3.6. Attendance Reports Screen

*   **Elements**: Filter Options (Date Range, Course, Student ID), Report Type Selector (Daily, Weekly, Monthly, By Course, By Student), "Generate Report" Button, Report Display
*   **Functionality**: Allows faculty and administrators to generate and view various attendance reports based on selected criteria and export them for further analysis or recor
```

```
Database schema created successfully in an in-memory SQLite database.
Tables created: ['Students', 'Courses', 'StudentCourses', 'sqlite_sequence', 'FacialEmbeddings', 'Attendance', 'Users']
```

EXPLANATION

In this task, we used prompts to guide AI tools in designing the application.
The prompts helped generate system architecture, database schema, and UI wireframes.
These outputs provided a clear structure and design of the system.
Overall, it made the planning process faster and more efficient.

Task 3 – AI-Assisted Code Development

PROMPT

Generate a simple full-stack application for a student attendance tracking system. Include frontend using HTML, CSS, and JavaScript, backend using Python Flask with REST APIs, and database schema with SQL queries. Also include integration using API calls, basic authentication, and error handling. Keep the code simple, clear, and suitable for a student-level project.

OUTPUT

```
# Install necessary packages
!pip install Flask Flask-Cors werkzeug
```

```
Requirement already satisfied: Flask in /usr/local/lib/python3.12/dist-packages (3.1.3)
Collecting Flask-Cors
  Downloading flask_cors-6.0.2-py3-none-any.whl.metadata (5.3 kB)
Requirement already satisfied: werkzeug in /usr/local/lib/python3.12/dist-packages (3.1.6)
Requirement already satisfied: blinker>=1.9.0 in /usr/local/lib/python3.12/dist-packages (from Flask) (1.9.0)
Requirement already satisfied: click>=8.1.3 in /usr/local/lib/python3.12/dist-packages (from Flask) (8.3.1)
Requirement already satisfied: itsdangerous>=2.2.0 in /usr/local/lib/python3.12/dist-packages (from Flask) (2.2.0)
Requirement already satisfied: jinja2>=3.1.2 in /usr/local/lib/python3.12/dist-packages (from Flask) (3.1.6)
Requirement already satisfied: markupsafe>=2.1.1 in /usr/local/lib/python3.12/dist-packages (from Flask) (3.0.3)
Downloading flask_cors-6.0.2-py3-none-any.whl (13 kB)
Installing collected packages: Flask-Cors
Successfully installed Flask-Cors-6.0.2
```

```
# Create app.py for the Flask Backend
%%writefile app.py
import sqlite3
from flask import Flask, request, jsonify, g, send_from_directory
from werkzeug.security import generate_password_hash, check_password_hash
from flask_cors import CORS
import os

# --- Flask App Setup ---
app = Flask(__name__)
CORS(app) # Enable CORS for all routes
app.config['DATABASE'] = 'attendance.db'
app.config['SECRET_KEY'] = 'your_strong_secret_key_here' # IMPORTANT: Change this in production!

# --- Database Utility Functions ---
def get_db():
    db = getattr(g, '_database', None)
```

```
# --- Database Utility Functions ---
def get_db():
    db = getattr(g, '_database', None)
    if db is None:
        db = g._database = sqlite3.connect(app.config['DATABASE'])
        db.row_factory = sqlite3.Row # This makes rows behave like dictionaries
    return db

@app.teardown_appcontext
def close_connection(exception):
    db = getattr(g, '_database', None)
    if db is not None:
        db.close()

def init_db():
    with app.app_context():
        db = get_db()
        cursor = db.cursor()
        # SQL DDL statements for creating tables (reusing the schema from before)
        sql_schema = """
        CREATE TABLE IF NOT EXISTS Students (
            student_id VARCHAR(10) PRIMARY KEY,
            first_name VARCHAR(50) NOT NULL,
            last_name VARCHAR(50) NOT NULL,
            email VARCHAR(100) UNIQUE NOT NULL,
            date_of_birth DATE,
            enrollment_date DATE NOT NULL,
            created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
        );

        CREATE TABLE IF NOT EXISTS Courses (
            course_id VARCHAR(10) PRIMARY KEY,
```

```
        CREATE TABLE IF NOT EXISTS StudentCourses (
            student_course_id INTEGER PRIMARY KEY AUTOINCREMENT,
            student_id VARCHAR(10) NOT NULL,
            course_id VARCHAR(10) NOT NULL,
            FOREIGN KEY (student_id) REFERENCES Students(student_id) ON DELETE CASCADE,
            FOREIGN KEY (course_id) REFERENCES Courses(course_id) ON DELETE CASCADE,
            UNIQUE (student_id, course_id)
        );

        CREATE TABLE IF NOT EXISTS FacialEmbeddings (
            embedding_id INTEGER PRIMARY KEY AUTOINCREMENT,
            student_id VARCHAR(10) UNIQUE NOT NULL,
            embedding_vector BLOB NOT NULL,
            image_path VARCHAR(255),
            created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
            FOREIGN KEY (student_id) REFERENCES Students(student_id) ON DELETE CASCADE
        );

        CREATE TABLE IF NOT EXISTS Attendance (
            attendance_id INTEGER PRIMARY KEY AUTOINCREMENT,
            student_id VARCHAR(10) NOT NULL,
            course_id VARCHAR(10) NOT NULL,
            timestamp TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP,
            status VARCHAR(20) DEFAULT 'Present',
            FOREIGN KEY (student_id) REFERENCES Students(student_id) ON DELETE CASCADE,
            FOREIGN KEY (course_id) REFERENCES Courses(course_id) ON DELETE CASCADE
        );

        CREATE TABLE IF NOT EXISTS Users (
            user_id INTEGER PRIMARY KEY AUTOINCREMENT,
            username VARCHAR(50) UNIQUE NOT NULL,
            password_hash VARCHAR(255) NOT NULL,
            email VARCHAR(100) UNIQUE NOT NULL,
            role VARCHAR(20) NOT NULL DEFAULT 'Faculty',
            created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
        );
        """
        cursor.executescript(sql_schema)
        db.commit()
        print("Database schema created or updated successfully.")

        # Add a default admin user if one doesn't exist for testing
        cursor.execute("SELECT * FROM Users WHERE username = ?", ('admin',))
        if not cursor.fetchone():
            hashed_password = generate_password_hash('password') # Default password 'password'
            cursor.execute(
                "INSERT INTO Users (username, password_hash, email, role) VALUES (?, ?, ?, ?)",
                ('admin', hashed_password, 'admin@example.com', 'Admin')
            )
            db.commit()
            print("Default admin user 'admin' with password 'password' added.")

# Initialize the database when the app starts
with app.app_context():
    init_db()

# --- API Endpoints ---

# Serve static files (HTML, CSS, JS)
@app.route('/')
def serve_index():
    return send_from_directory('.', 'index.html')

@app.route('/<path:path>')
def serve_static(path):
    return send_from_directory('.', path)

# User Login
@app.route('/api/login', methods=['POST'])
```

```
        password = request.json.get('password')

        db = get_db()
        user = db.execute("SELECT * FROM Users WHERE username = ?", (username,)).fetchone()

        if user and check_password_hash(user['password_hash'], password):
            # In a real app, you'd generate and return a token (JWT) here
            return jsonify({'message': 'Login successful', 'user_id': user['user_id'], 'username': user['username'], 'role': user['role']}), 200
        else:
            return jsonify({'message': 'Invalid credentials'}), 401

    # Get all students or add a new student
    @app.route('/api/students', methods=['GET', 'POST'])
    def students():
        db = get_db()
        if request.method == 'POST':
            data = request.json
            student_id = data.get('student_id')
            first_name = data.get('first_name')
            last_name = data.get('last_name')
            email = data.get('email')
            date_of_birth = data.get('date_of_birth')
            enrollment_date = data.get('enrollment_date') or str(data.get('created_at')) # Use created_at if enrollment_date not provided, or default

            if not all([student_id, first_name, last_name, email]):
                return jsonify({'message': 'Missing required student data'}), 400

            try:
                cursor = db.cursor()
                cursor.execute(
                    "INSERT INTO Students (student_id, first_name, last_name, email, date_of_birth, enrollment_date) VALUES (?, ?, ?, ?, ?, ?)",
                    (student_id, first_name, last_name, email, date_of_birth, enrollment_date)
                )
                db.commit()
                return jsonify({'message': 'Student added successfully', 'student_id': student_id}), 201
            except sqlite3.IntegrityError as e:
                if "UNIQUE constraint failed: Students.student_id" in str(e):
                    return jsonify({'message': 'Student with this ID already exists'}), 409
                elif "UNIQUE constraint failed: Students.email" in str(e):
                    return jsonify({'message': 'Student with this email already exists'}), 409
                else:
                    return jsonify({'message': f'Database error: {str(e)}'}), 500
            except Exception as e:
                return jsonify({'message': f'Error adding student: {str(e)}'}), 500

        else: # GET request
            students_data = db.execute("SELECT * FROM Students").fetchall()
            return jsonify([dict(student) for student in students_data]), 200

    # Get all courses or add a new course
    @app.route('/api/courses', methods=['GET', 'POST'])
    def courses():
        db = get_db()
        if request.method == 'POST':
            data = request.json
            course_id = data.get('course_id')
            course_name = data.get('course_name')
            course_code = data.get('course_code')
            instructor_id = data.get('instructor_id')
            description = data.get('description')

            if not all([course_id, course_name, course_code]):
                return jsonify({'message': 'Missing required course data'}), 400
            try:
                cursor = db.cursor()
                cursor.execute(
                    "INSERT INTO Courses (course_id, course_name, course_code, instructor_id, description) VALUES (?, ?,
```

```
print("Files app.py, index.html, style.css, and script.js have been created.")
print("Please follow the instructions in this cell to run the application.")
```

```
Files app.py, index.html, style.css, and script.js have been created.
Please follow the instructions in this cell to run the application.
```

```
!python app.py &
```

```
Database schema created or updated successfully.
 * Serving Flask app 'app'
 * Debug mode: on
WARNING: This is a development server. Do not use it in a production deployment. Use a production WSGI server instead.
 * Running on all addresses (0.0.0.0)
 * Running on http://127.0.0.1:5000
 * Running on http://172.28.0.12:5000
Press CTRL+C to quit
```

EXPLANATION

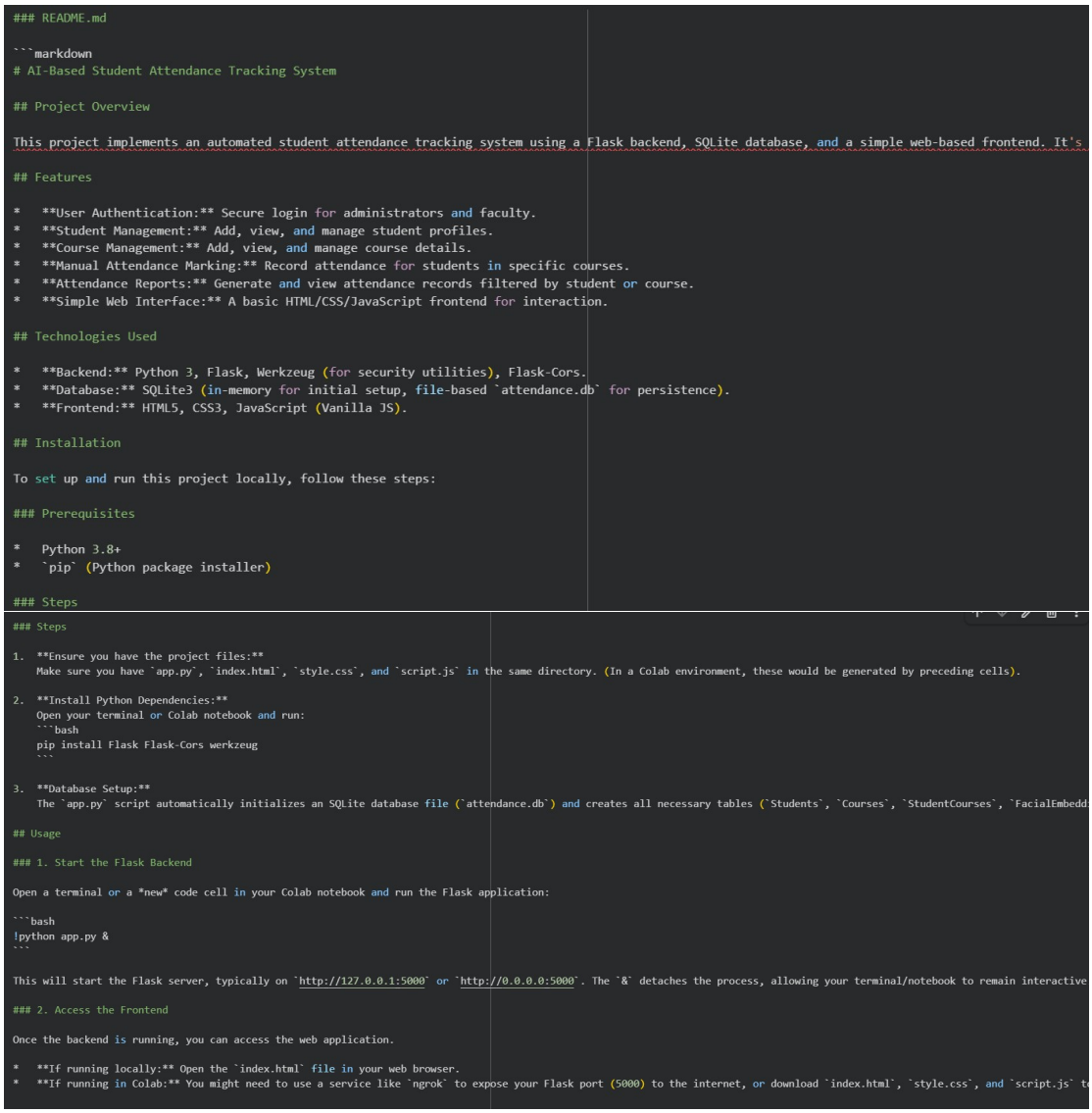
In this task, a single prompt was used to generate the complete application code including frontend, backend, and database. The AI helped in writing and connecting all parts of the system. This made development faster and easier. The generated code was then reviewed and understood before use.

Task 4 – Documentation & Code Quality

PROMPT

Generate documentation and inline comments for the given project code. Also create a README file with project description, installation steps, and usage instructions. Review the code and suggest improvements for better readability and maintainability.

OUTPUT



````
### README.md

```markdown
# AI-Based Student Attendance Tracking System

## Project Overview

This project implements an automated student attendance tracking system using a Flask backend, SQLite database, and a simple web-based frontend. It's

## Features

*   **User Authentication:** Secure login for administrators and faculty.
*   **Student Management:** Add, view, and manage student profiles.
*   **Course Management:** Add, view, and manage course details.
*   **Manual Attendance Marking:** Record attendance for students in specific courses.
*   **Attendance Reports:** Generate and view attendance records filtered by student or course.
*   **Simple Web Interface:** A basic HTML/CSS/JavaScript frontend for interaction.

## Technologies Used

*   **Backend:** Python 3, Flask, Werkzeug (for security utilities), Flask-Cors.
*   **Database:** SQLite3 (in-memory for initial setup, file-based `attendance.db` for persistence).
*   **Frontend:** HTML5, CSS3, JavaScript (Vanilla JS).

## Installation

To set up and run this project locally, follow these steps:

### Prerequisites

*   Python 3.8+
*   `pip` (Python package installer)

### Steps
````

````
### Steps

1.  **Ensure you have the project files:**
    Make sure you have `app.py`, `index.html`, `style.css`, and `script.js` in the same directory. (In a Colab environment, these would be generated by preceding cells).

2.  **Install Python Dependencies:**
    Open your terminal or Colab notebook and run:
    ```bash
    pip install Flask Flask-Cors werkzeug
    ```

3.  **Database Setup:**
    The `app.py` script automatically initializes an SQLite database file (`attendance.db`) and creates all necessary tables (`Students`, `Courses`, `StudentCourses`, `FacialEmbedd

## Usage

### 1. Start the Flask Backend

Open a terminal or a *new* code cell in your Colab notebook and run the Flask application:

```bash
!python app.py &
```

This will start the Flask server, typically on `http://127.0.0.1:5000` or `http://0.0.0.0:5000`. The `&` detaches the process, allowing your terminal/notebook to remain interactive

### 2. Access the Frontend

Once the backend is running, you can access the web application.

*   **If running locally:** Open the `index.html` file in your web browser.
*   **If running in Colab:** You might need to use a service like `ngrok` to expose your Flask port (5000) to the internet, or download `index.html`, `style.css`, and `script.js` t
````
````

```
### 3. Login to the System

Use the default administrator credentials to log in:

*   **Username:** `admin`
*   **Password:** `password`

### 4. Navigate and Manage

After logging in, you can use the navigation bar to:

*   **Dashboard:** View system statistics (e.g., total students, total courses).
*   **Students:** Add new students (Student ID, Name, Email, DOB) and view the student list.
*   **Courses:** Add new courses (Course ID, Name, Code, Instructor, Description) and view the course list.
*   **Attendance:** Manually mark attendance for students in courses. You will need to select a student, course, and status (Present, Absent, Late).
*   **Reports:** Generate attendance reports filtered by student or course.

## Future Enhancements

*   **Facial Recognition Integration:** Connect with a real-time facial recognition service (e.g., using OpenCV, dlib/FaceNet) to automate attendance marking.
*   **Improved Authentication:** Implement JWT-based authentication for enhanced security.
*   **User Roles & Permissions:** More granular control over what different user roles (Admin, Faculty) can do.
*   **Real-time Monitoring Interface:** A dedicated screen to show live camera feeds and recognized students.
*   **Advanced Reporting:** More sophisticated reports and analytics.
*   **Error Handling & Validation:** Robust client-side and server-side validation.
*   **Scalability:** Migrate to a more production-ready database (like PostgreSQL) and deploy on cloud platforms.
```

EXPLANATION

In this task, AI was used to generate documentation and comments for the code. A README file was created for project setup and usage. AI also helped in reviewing and improving the code quality. This made the project more readable and maintainable.

Task 5 – Testing & Debugging

PROMPT

Generate unit test cases for the given application code. Identify possible bugs, vulnerabilities, and security issues. Also suggest improvements to optimize performance and enhance code efficiency. Provide clear explanations for each suggestion in a simple and structured format suitable for a student project.

CODE

```python
#Generate unit test cases for the given application code. Identify possible bugs, vulnerabilities, ar
# Application Code: AI-Based Student Attendance Tracking System
import cv2
import numpy as np
import face_recognition
import os
from datetime import datetime
class AttendanceTracker:
    def __init__(self, known_faces_dir):
        self.known_faces = []
        self.known_names = []
        self.load_known_faces(known_faces_dir)

    def load_known_faces(self, directory):
        for filename in os.listdir(directory):
            if filename.endswith('.jpg') or filename.endswith('.png'):
                image = face_recognition.load_image_file(os.path.join(directory, filename))
                encoding = face_recognition.face_encodings(image)[0]
                self.known_faces.append(encoding)
                self.known_names.append(os.path.splitext(filename)[0])

    def mark_attendance(self, name):
        with open('attendance.csv', 'a') as f:
            now = datetime.now()
            dt_string = now.strftime('%Y-%m-%d %H:%M:%S')
            f.write(f'{name},{dt_string}\n')

    def recognize_faces(self, frame):
        rgb_frame = cv2.cvtColor(frame, cv2.COLOR_BGR2RGB)
        face_locations = face_recognition.face_locations(rgb_frame)
        face_encodings = face_recognition.face_encodings(rgb_frame, face_locations)

        for (top, right, bottom, left), face_encoding in zip(face_locations, face_encodings):
```

OUTPUT

```
# Possible Bugs, Vulnerabilities, and Security Issues:
1. The application does not handle cases where no faces are detected in the image, which could
   lead to an IndexError when trying to access face_encodings[0].
2. The application does not validate the input images, which could lead to security vulnerabilities if malicious files are uploaded.
3. The attendance.csv file is opened in append mode without any locking mechanism, which could lead to data corruption if multiple instances of
the application are running simultaneously.
# Suggestions for Improvements:
1. Add error handling to manage cases where no faces are detected in the image, preventing potential crashes.
2. Implement input validation to ensure that only valid image files are processed, enhancing security.
3. Use a database instead of a CSV file for storing attendance records, which can handle concurrent access and improve performance.
4. Optimize the face recognition process by using a more efficient algorithm or reducing the number of known faces to compare against, which can
enhance performance.
5. Implement a user authentication mechanism to restrict access to the attendance tracking system, enhancing security.
6. Consider using a more secure method for storing known face encodings, such as encryption, to protect sensitive data.
```

EXPLANATION

In this task, AI tools were used to test and debug the application. The AI helped generate unit test cases to check if the code works correctly. It also identified possible bugs and security issues. Additionally, AI suggested improvements to optimize performance and make the code more efficient.

Task 6 – Deployment & Presentation

PROMPT

Generate a presentation outline for an AI-based student attendance tracking system. Include problem statement, solution architecture, live demo explanation, and reflection on AI assistance and its limitations. Keep the content simple and suitable for a student project presentation.

OUTPUT

```
# Presentation Outline: AI-Based Student Attendance Tracking System
1. Introduction
   - Brief overview of the importance of attendance tracking in educational institutions.
2. Problem Statement
   - Challenges of manual attendance tracking (time-consuming, prone to errors, and lack of real-time data).
3. Solution Architecture
   - Overview of the AI-based attendance tracking system.
   - Components of the system (e.g., facial recognition, database management, user interface).
4. Live Demo Explanation
   - Step-by-step walkthrough of the system in action.
   - Demonstration of how the system captures attendance using facial recognition and updates the database in real-time.
5. Reflection on AI Assistance and Its Limitations
   - Benefits of using AI for attendance tracking (accuracy, efficiency, and real-time data).
   - Limitations of the system (privacy concerns, potential biases in facial recognition, and the need for a large dataset for training).
6. Conclusion
   - Summary of the key points discussed in the presentation.
   - Future improvements and potential applications of AI in education.
7. Q&A Session
   - Open the floor for questions and discussions with the audience.
# Note: The presentation should include visual aids such as diagrams of the system architecture, screenshots of the live demo, and charts to
illustrate the benefits and limitations of AI in attendance tracking.
```

EXPLANATIONIn this task, the application was deployed on a cloud platform so it can be accessed online. A final presentation was prepared explaining the problem, solution architecture, and live demo. AI tools helped in creating content and improving the presentation. We also discussed the limitations of AI used in the project.